Лабораторная работа № 4
Измерение латентности кэш-памяти процессора и оперативной памяти для заданной вычислительной системы

Цель работы: познакомиться с базовыми принципами определения пропускной способности подсистемы памяти CPU.

Более 40 лет назад в 1980-е годы появилась известная проблема быстродействия динамической памяти (DRAM), выражающаяся в том, что при переходе на CMOS-технологию изготовления транзисторов в совокупности с уменьшением их физического размера процессоры начали активно повышать свое быстродействие (тактовую частоту, достигающую сегодня величин 3–5 ГГц), в то время как динамическая память остановилась на значениях около 100 МГц ввиду достижения физического предела из-за ограниченного времени перезаряда конденсаторов в ячейках. Следствием этого явилось появление в составе процессоров статической памяти на базе триггеров (SRAM), организованной в виде кэш-памяти, имеющей существенно меньший объем (мегабайты у SRAM против гигабайт у DRAM), но взамен работающей на тактовой частоте процессора. Ввиду наличия зависимости задержки доступа к данным в кэш-памяти от ее объема в процессорах кэш-память обычно разбивается на уровни (англ. Levels), обозначаемые как L1, L2 и т.д. Подсистема памяти при этом образует т.н. «пирамиду памяти», в состав которой сверху-вниз входят:

- регистры процессора;
- кэш-память процессора различного уровня (L1–L4);
- оперативная память (DRAM);
- внешняя память (SSD, HDD, RAID-массивы, NAS/SAN накопители).

В составе некоторых вычислительных систем могут присутствовать дополнительные промежуточные уровни, например, MRAM- или HBM-память. Обрабатываемая порция данных попадает в один из уровней памяти в составе пирамиды памяти, а задержка доступа к ней оказывает существенное влияние на итоговое время обработки данных.

Современные CPU семейства x86 включают в своем составе от 2 до 4 уровней кэш-памяти с различной организацией (кэш команд/кэш данных/унифицированный кэш, кэш прямого отображения/полно-ассоциативный или наборно-ассоциативный кэш, инклюзивный или эксклюзивный кэш, политики сквозной записи и вытеснения данных из кэш-памяти и т.д.) в зависимости от модели. Различные кэши характеризуются различной величиной задержки доступа к данным или латентностью (англ. latency). Время работы программ, критичных к темпу поступления данных из памяти, зачастую лимитируется именно этой величиной и зависит от объема обрабатываемых данных, что определяет их расположение в кэше одного из уровней. Работа подсистемы кэш-памяти прозрачна для программиста, о ее наличии можно судить по косвенным признакам, к которым относится время выполнения типового фрагмента кода, требующего доступа к данным.

Исследование задержек доступа к данным в кэш-памяти и оперативной памяти может быть измерено путем решения тестовой задачи нахождения суммы элементов массива $S = \sum_{i=1}^{N} a_i$, тип данных в рассмотренных ниже примерах – целые числа размером 4 байта. Операция целочисленного сложения выполняется на современных процессорах за

1 такт, время работы лимитируется темпом поступления данных в АЛУ процессора, размер массива влияет на его расположение в кэш-памяти различного уровня.

Тестирование латентностей доступа в рассмотренных ниже примерах было выполнено на следующих процессорах со следующей конфигурацией кэш-памяти и оперативной памяти.

1. Intel Atom N270 – 24 КБ L1d, 512 КБ L2, DDR2.
2. Intel Core i7 4770 – 64 КБ L1d, 256 КБ L2, 8 МБ L3, DDR3.
3. AMD Ryzen 7 7700 – 64 КБ L1d, 1 МБ L2, 32 МБ L3, DDR5.

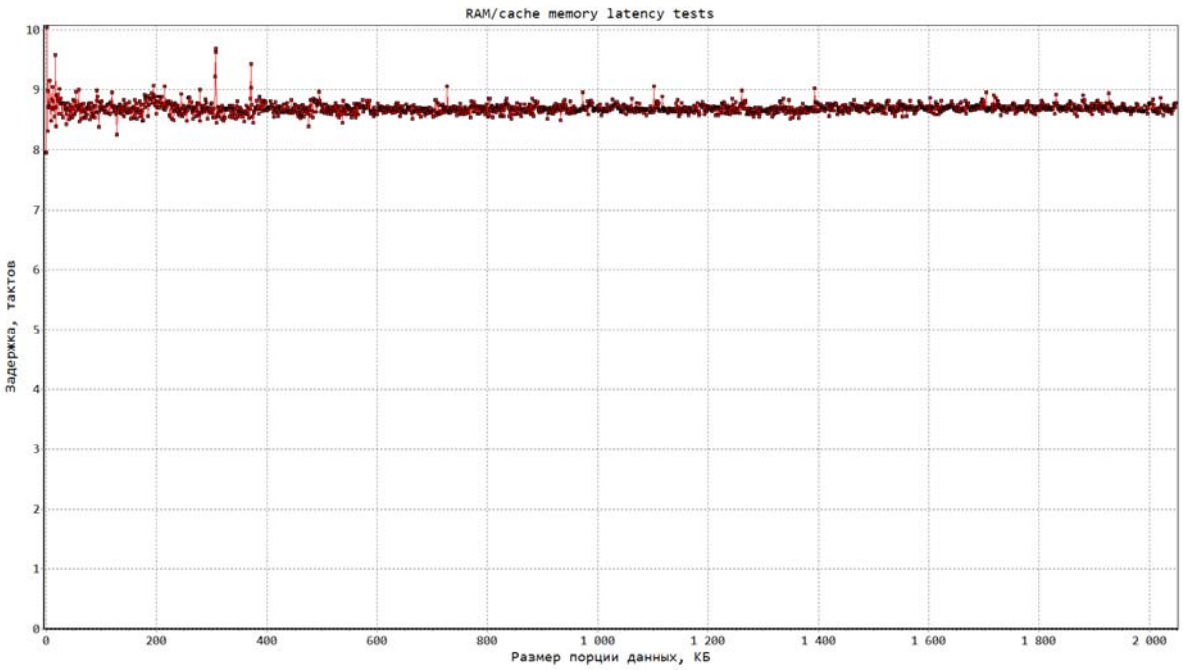
Простейшим тестом является сложение элементов массива подряд в порядке возрастания значений индексов (S := S + a[1], S := S + a[2], …, S := S + a[N]):

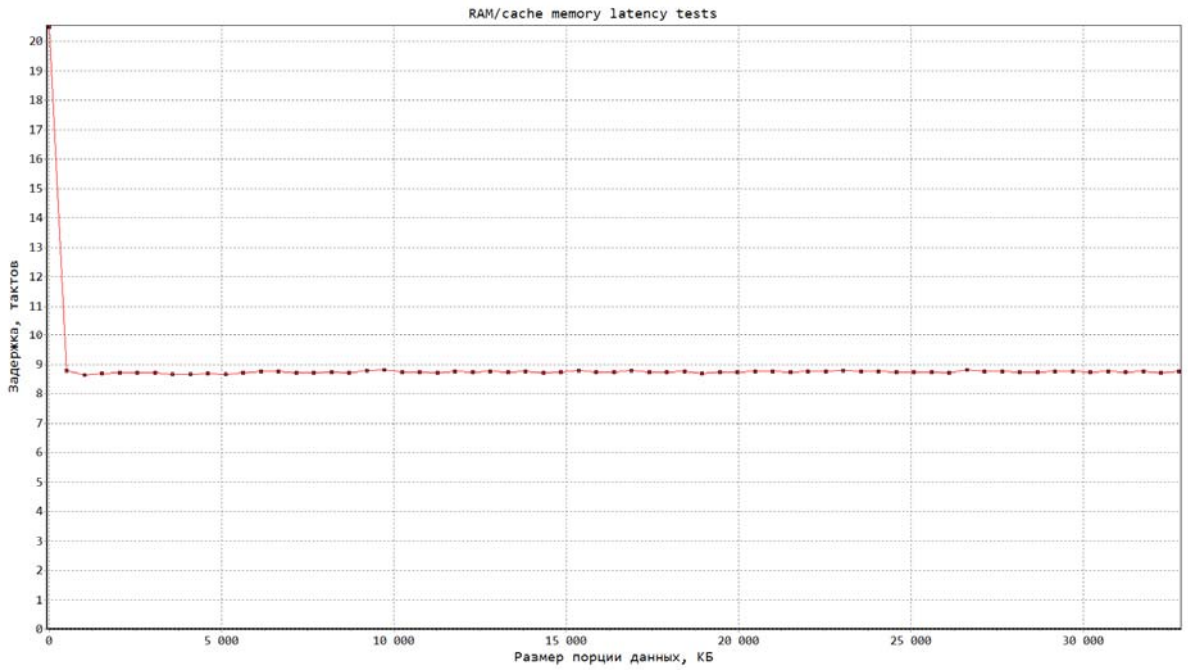
```
for i from 1 to n do
    S := S + a[i];
```

При этом необходимо учитывать следующие эффекты, являющиеся следствием логики работы процессоров современных поколений.

1. Обмен данными между памятью и кэш-памятью или между различным уровнями кэш-памяти производится порциями размером 64 байта (размер кэш-линии в кэш-памяти, англ. cache line size или chunk). При обращении к первым 4 байтам, отсутствующим в кэше, кэш-линия размером 64 байта однократно загружается из оперативной памяти в кэш-память, остальные обращения $(64/4-1=15)$ производятся к ней, что снижает достигаемую эффективную задержку доступа к данным.
2. При обращении в память по последовательным адресам современные процессоры эффективно задействуют механизм аппаратной предвыборки (англ. hardware prefetch): при выполнении обработки *i*-й порции данных (*i*-й кэш-линии) следующая за ней $(i+1)$-я заблаговременно загружается в кэш до наличия обращений к ней, что также уменьшает эффективную задержку доступа к данным.

Примеры зависимости времени выполнения одной итерации цикла сложения с накоплением от размера массива *N* приведены на рис. 1.





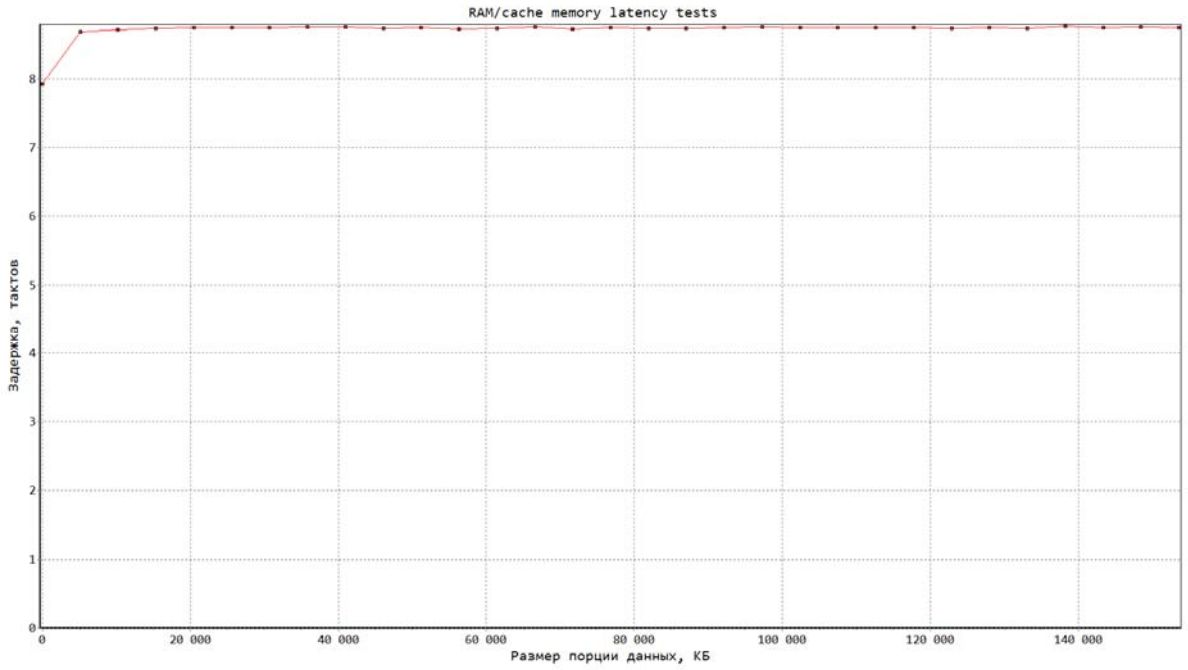


Рис. 1. Зависимости времени выполнения одной итерации цикла сложения с накоплением от размера обрабатываемых данных (размера массива *N*) на процессоре Intel Core i7 4770

Таблица 1. Эффективная задержка доступа к данным различного размера, последовательный порядок обращения, тактов

CPU	Задержка
Intel Atom N270	**29**
Intel Core i7 4770	**8–9**
AMD Ryzen 7 7700	**7–8**

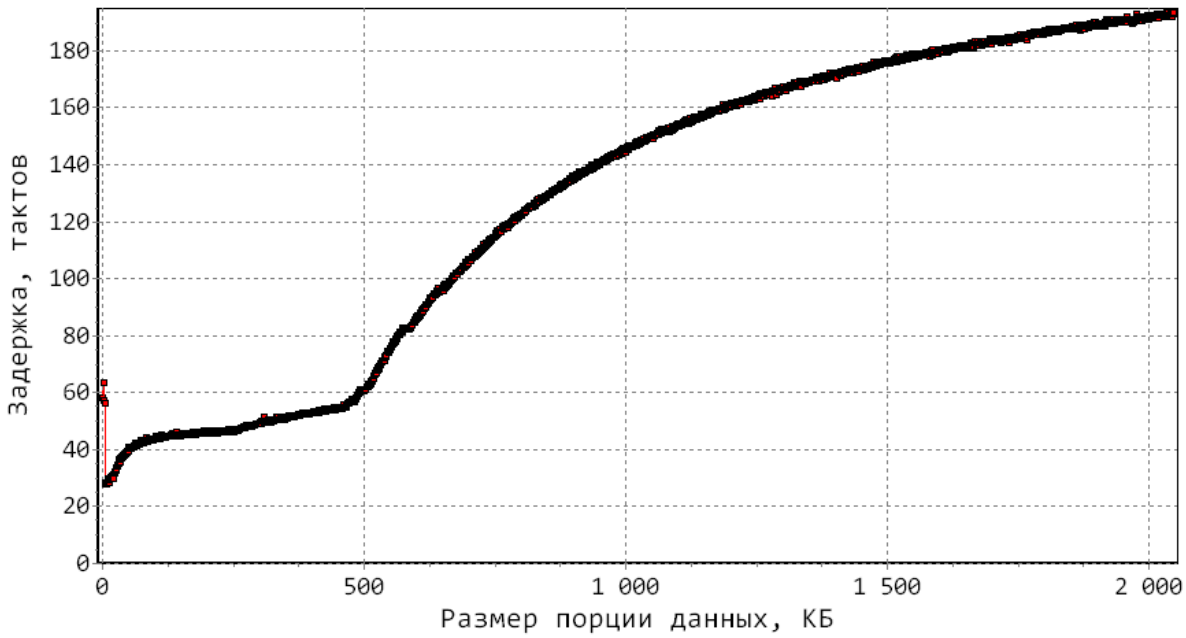
Анализ приведенных зависимостей позволяет сделать вывод о том, что в данной задаче темп обработки данных зависит от ряда факторов и не лимитируется напрямую латентностью доступа к кэш-памяти одного из уровней или к оперативной памяти ввиду рассмотренных выше причин.

Более сложным для процессора случаем являются обращения к памяти в случайном порядке.

```
for i from 1 to n do
{
    index := random(1, n);
    S := S + a[index];
}
```

При этом за одно обращение к подсистеме памяти (одну итерацию цикла) из загружаемой в кэш-память кэш-линии с большой вероятностью используется только 4 из 64 байт, механизм аппаратной предвыборки данных не работает ввиду отсутствия регулярного паттерна обращения к памяти (обращений по возрастающим или убывающим адресам), время одной итерации цикла сложения с накоплением определяется латентностью доступа к данным. При объеме данных обрабатываемого массива меньше размера кэша L1 он целиком будет размещен в нем, при большем объеме – часть обращений будет происходить в кэш L1, часть – в кэш L2, при еще большем – в кэш L3 и т.д., затем – в оперативную память при исчерпании объема кэша максимального уровня.

Примеры зависимости времени выполнения одной итерации цикла сложения с накоплением от объема обрабатываемых данных (размера массива *N*) приведены на рис. 2–4.



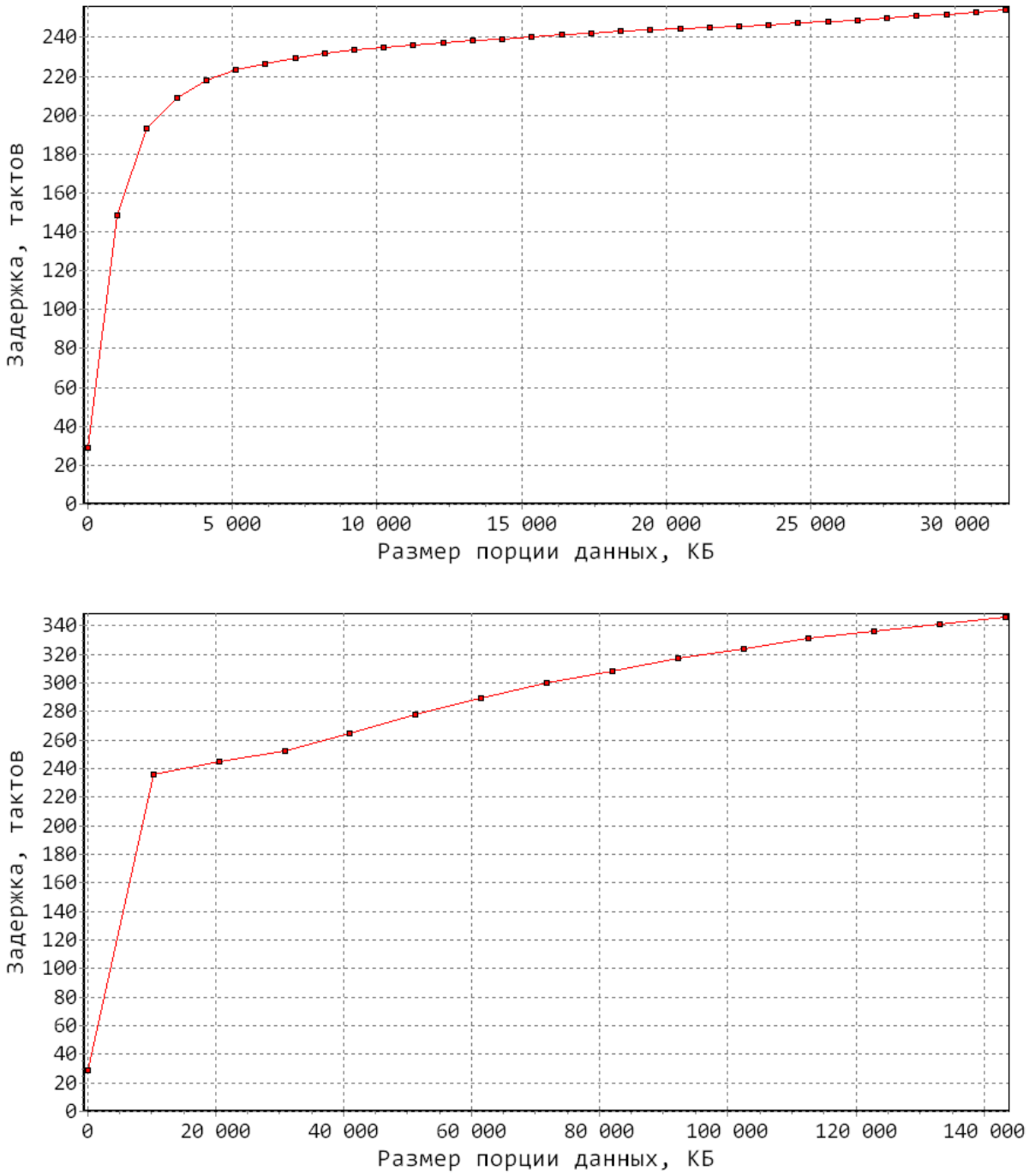
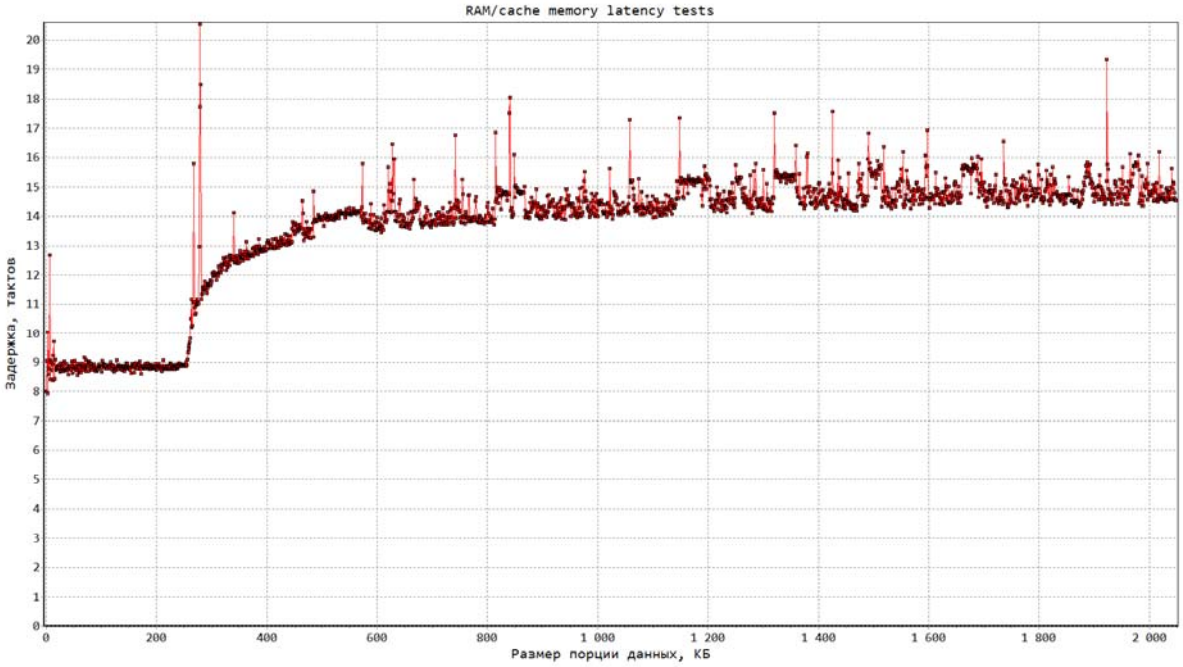
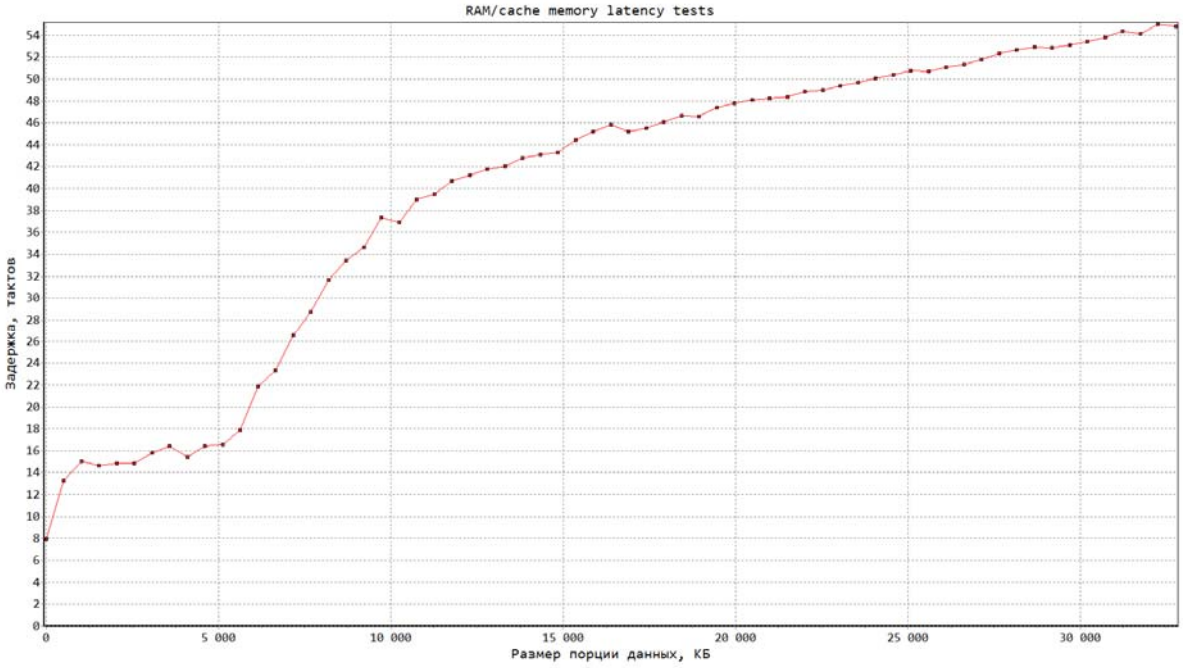


Рис. 2. Зависимость времени выполнения одной итерации сложения с накоплением от размера обрабатываемых данных (размера массива *N*), Intel Atom N270


RAM/cache memory latency tests
Задержка, тактов
Размер порции данных, КБ


RAM/cache memory latency tests
Задержка, тактов
Размер порции данных, КБ

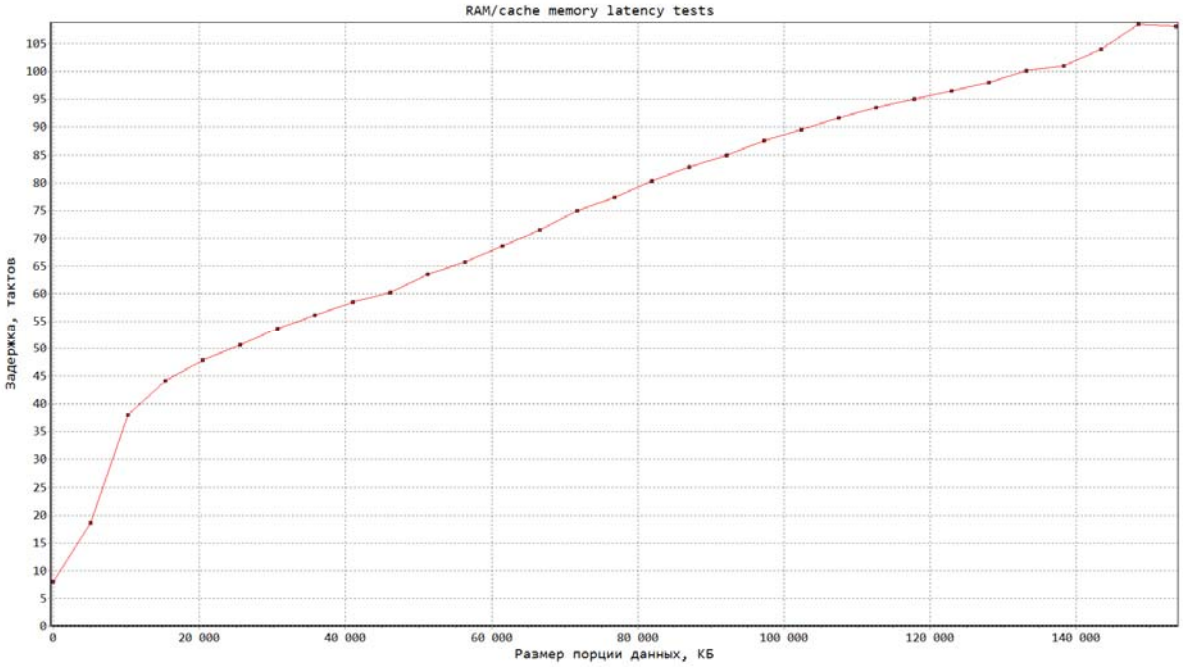
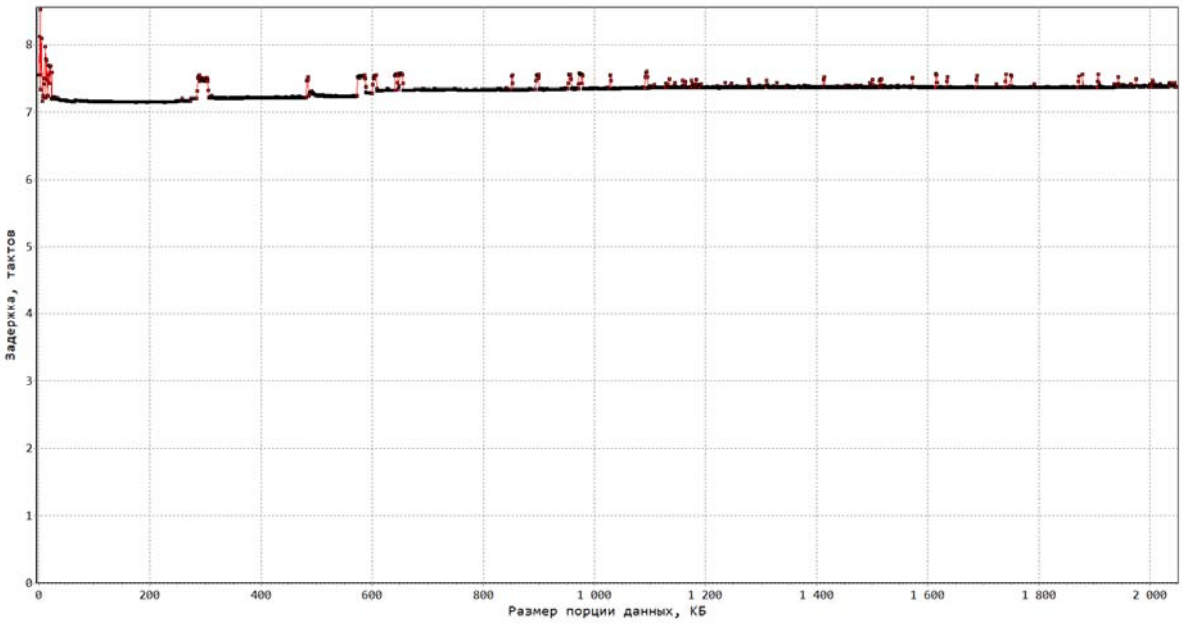


Рис. 3. Зависимость времени выполнения одной итерации сложения с накоплением от размера массива *N*, Intel Core i7 4770



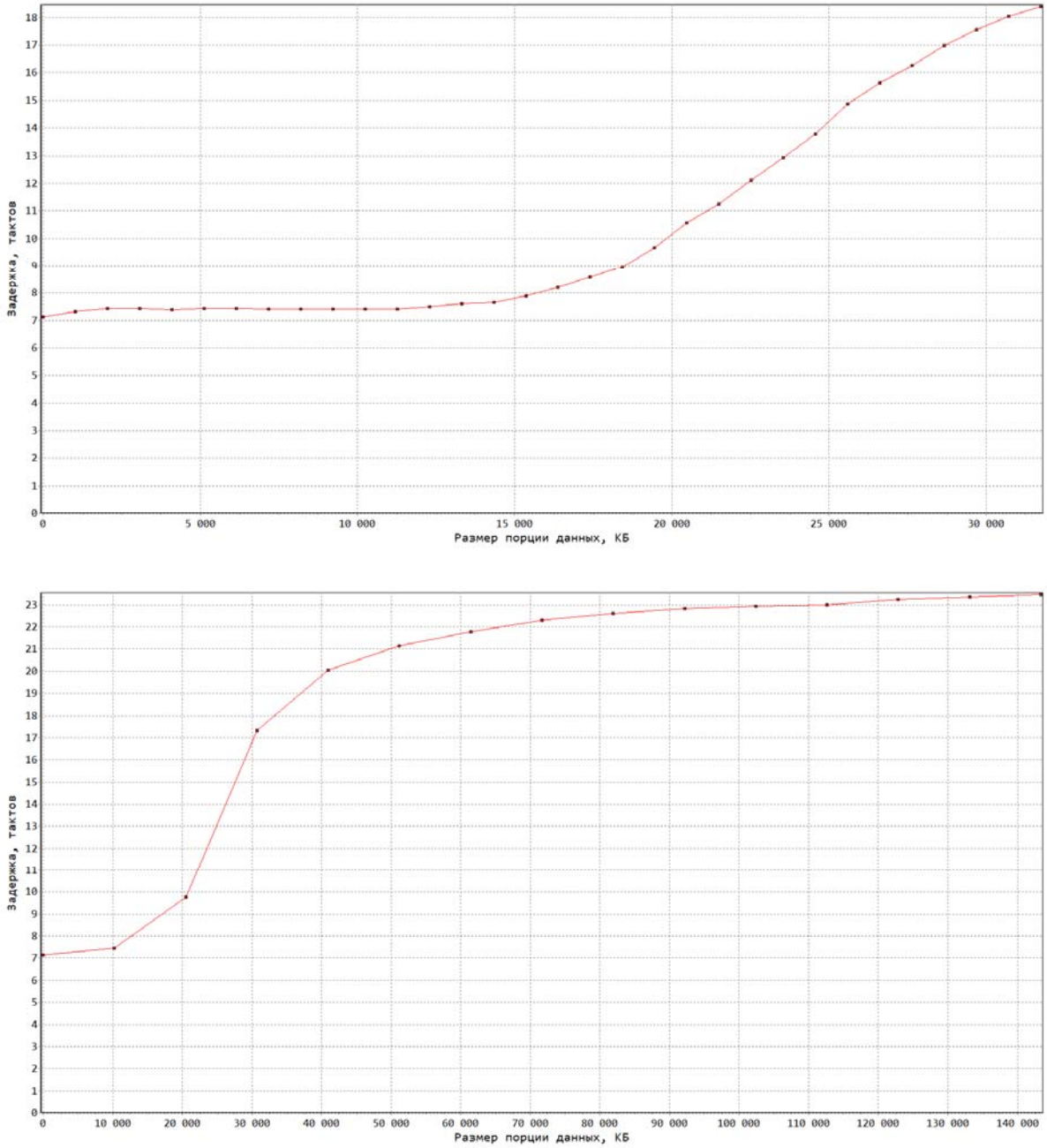


Рис. 4. Зависимость времени выполнения одной итерации сложения с накоплением от размера обрабатываемых данных (размера массива *N*), AMD Ryzen 7 7700

Таблица 2. Эффективная задержка доступа к данным различного размера, случайный порядок обращения, тактов

CPU	Кэш L1d	Кэш L2	Кэш L3	RAM
Intel Atom N270	**25–30**	**45** (объем данных до 256 КБ), **45–55** (объем данных до 512 КБ)	–	**240–340**

Intel Core i7 4770	**9**	**9**	**14–16**	**до 106**
AMD Ryzen 7 7700	**7**	**7**	**7–8** (объем данных до 16 МБ), **8–18** (объем данных до 32 МБ)	**23**

Замечание 1. При выполнении данных тестов наличие фоновой нагрузки на процессор может существенно повлиять на результаты (искажение графиков из-за наличия вычислительной нагрузки на другом логическом ядре, наличие в кэше данных «чужих» потоков, снижающих его эффективный объем и т.д., см. рис. 5).

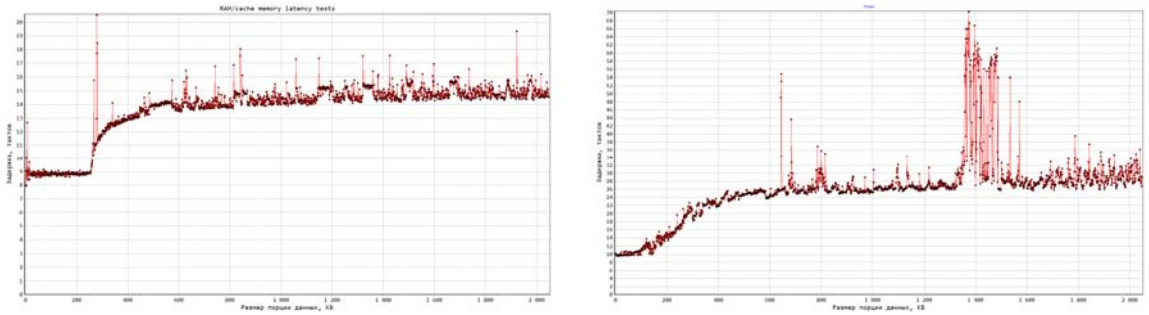


Рис. 5. Пример замеров длительности выполнения итерации, проведенных на «чистом» процессоре без фоновой нагрузки (слева) и при наличии фоновой нагрузки (справа)

Замечание 2. Рост задержки с ростом объема обрабатываемых данных зависит также от механизма трансляции виртуальных адресов оперативной памяти в физические с использованием буферов ассоциативной трансляции (сокр. TLB-буфферов, Translation Lookaside Buffers) в составе процессора, которые по принципу организации также являются специализированными кэшами.

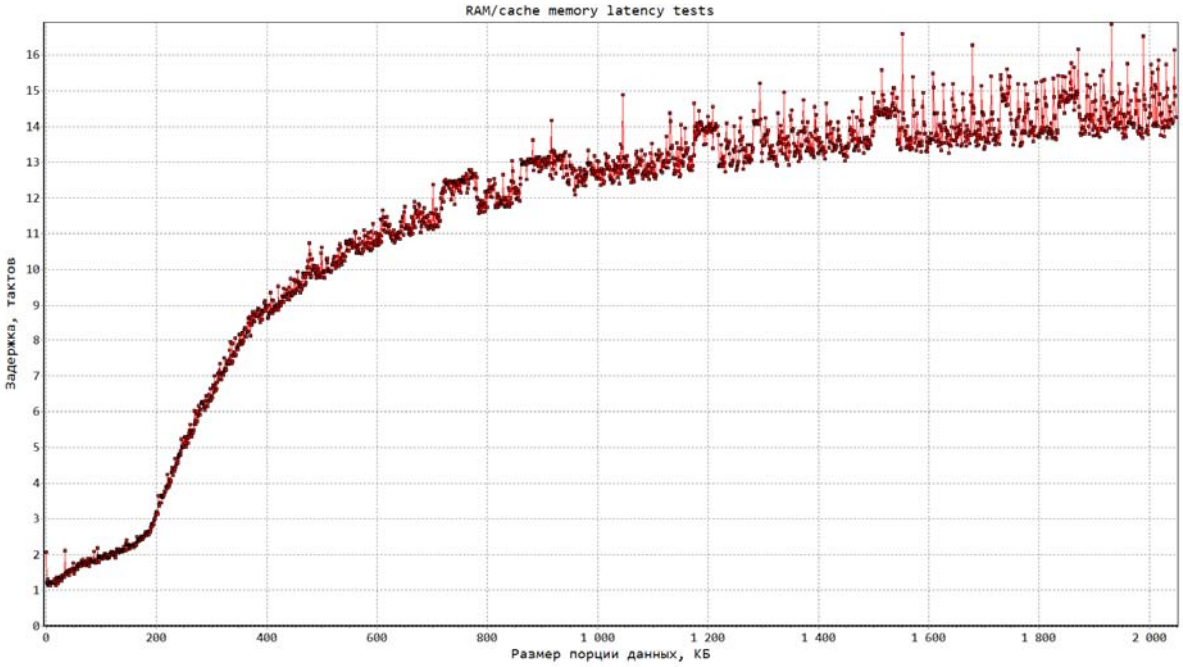
Полученные в результате замеров данные времен выполнения одной итерации цикла вычисления суммы с накоплением включают в своем составе не только задержки доступа к кэш-памяти или оперативной памяти при чтении значения `a[index]`, но и время работы функции, обеспечивающей получение следующего случайного индекса массива, т.е. являются несколько завышенными. Чтобы избежать этого и приблизиться к точному значению задержки доступа в кэш-память или оперативную память, можно заранее сформировать массив со случайными индексами элементов обрабатываемого массива, а затем использовать их в цикле нахождения суммы с накоплением.

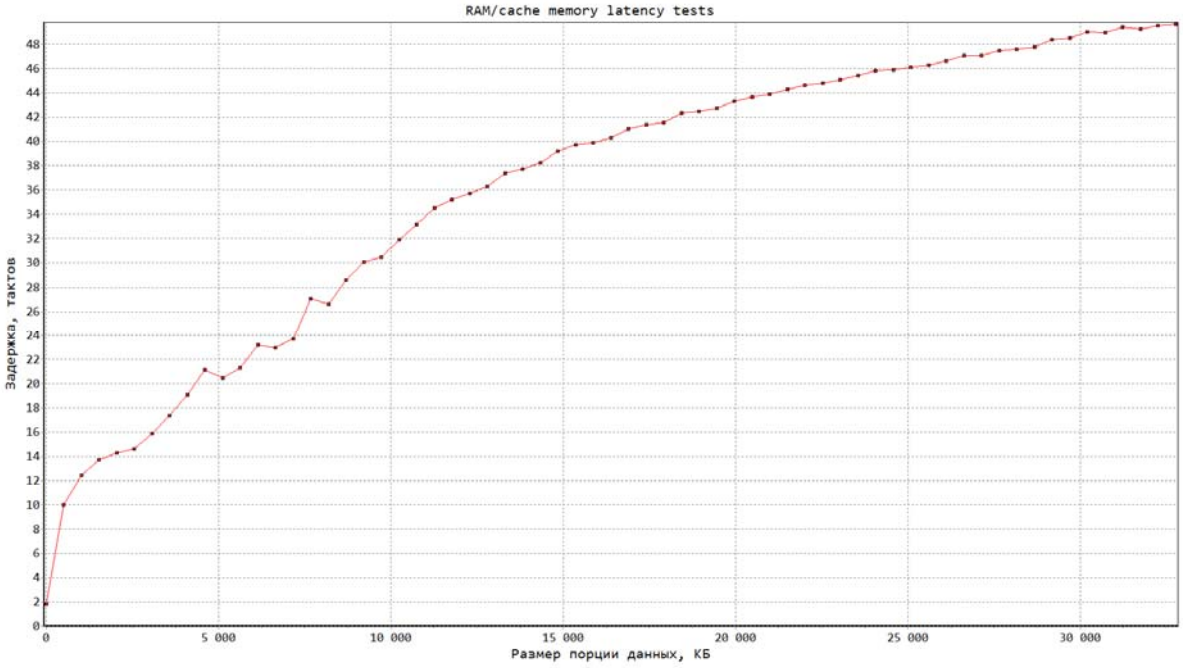
```
// Подготовительный цикл
for i from 1 to n do
    index_arr[i] := random(1, n);

// Рабочий цикл вычисления искомой суммы с накоплением
for i from 1 to n do
{
    index := index_arr[i];
    S := S + a[index];
}
```

Однако при этом хранение массива индексов `index_arr[]` также требует объема памяти, соизмеримого с объемом памяти для хранения обрабатываемого массива данных `a[]`, что несколько снижает эффективный объем кэшей. Примеры зависимости времени

выполнения одной итерации цикла сложения с накоплением от размера массива N при использовании дополнительного массива индексов приведены на рис. 6–8.





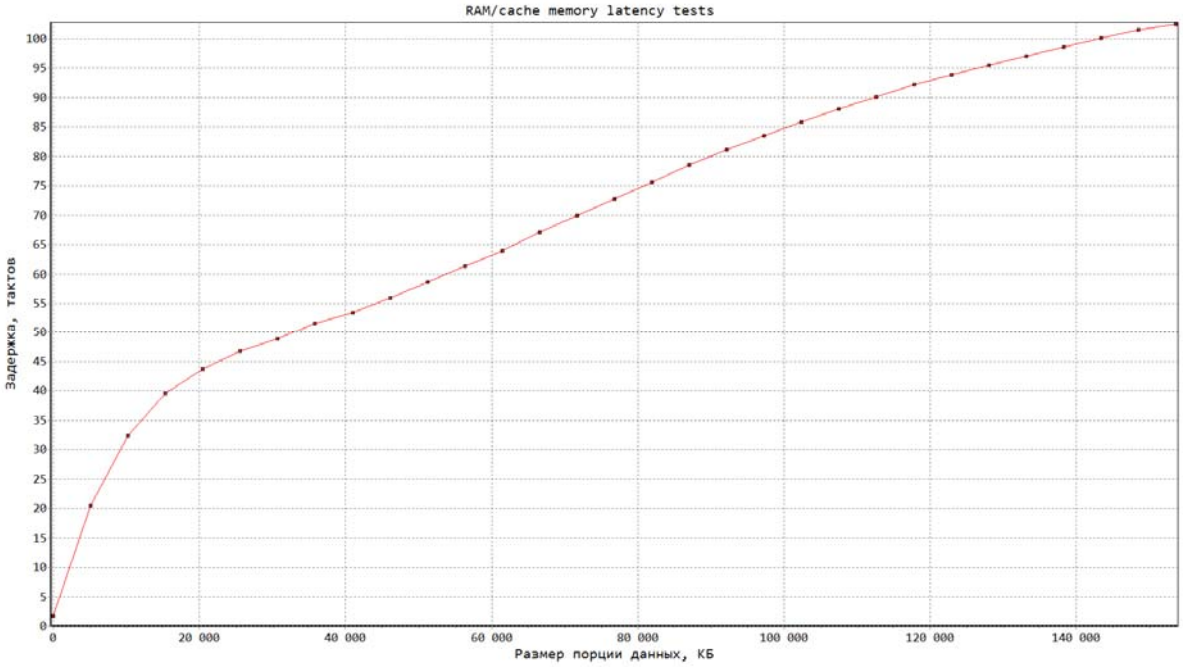
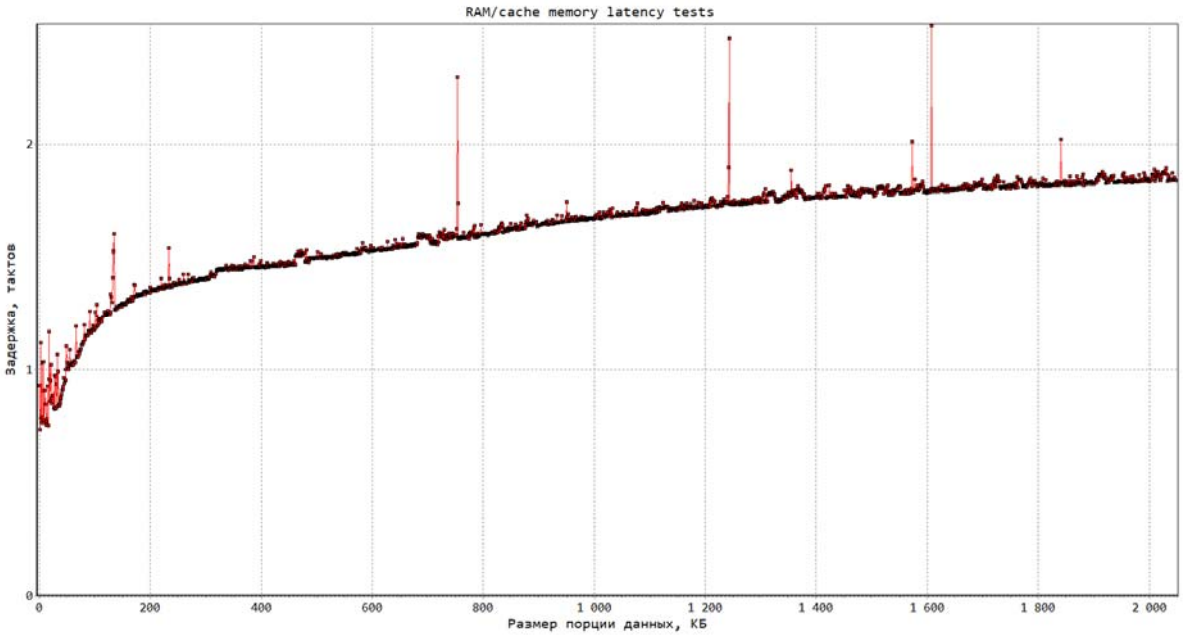
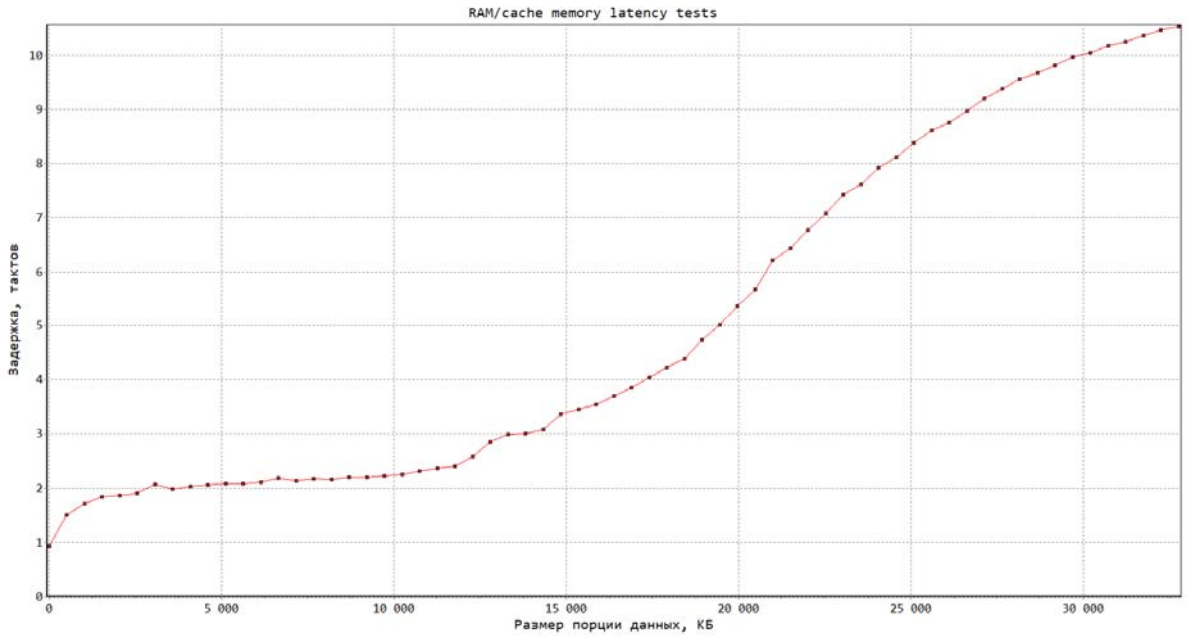


Рис. 7. Зависимость времени выполнения одной итерации сложения с накоплением от размера обрабатываемых данных (размера массива *N*), Intel Core i7 4770





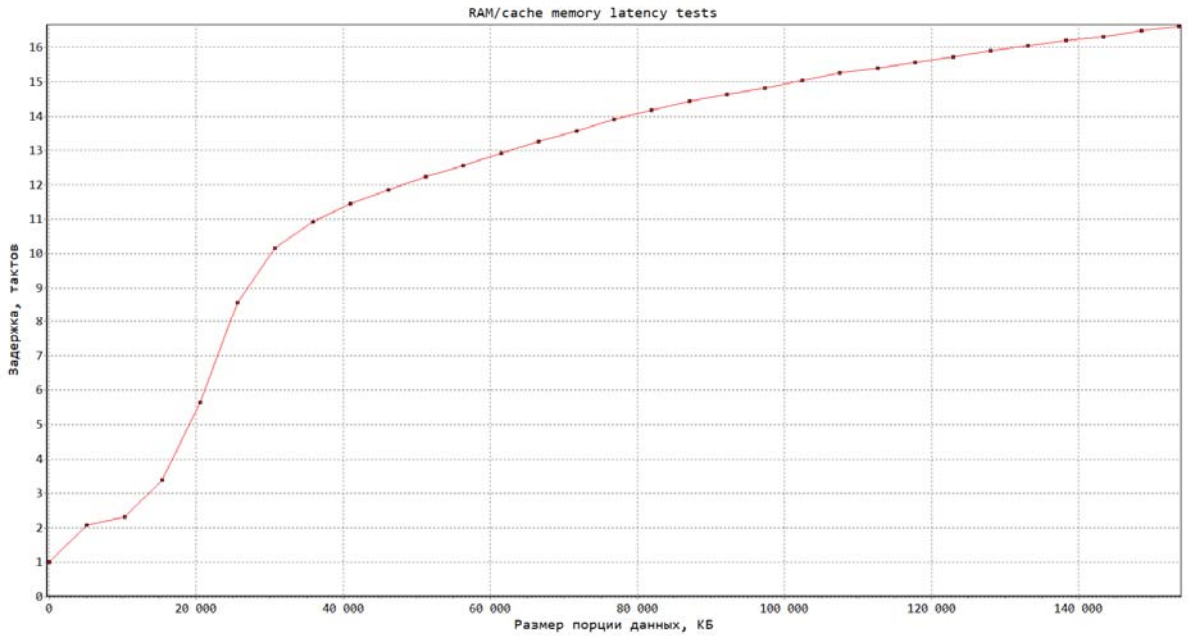


Рис. 8. Зависимость времени выполнения одной итерации сложения с накоплением от размера обрабатываемых данных (размера массива *N*), AMD Ryzen 7 7700

Таблица 3. Эффективная задержка доступа к данным различного размера, случайный порядок обращения с использованием массива индексов, тактов

CPU	Кэш L1d	Кэш L2	Кэш L3	RAM
Intel Atom N270	**6**	**26** (объем данных до 256 КБ)	–	**до 210**
Intel Core i7 4770	**1**	**2–3** (объем данных до 192 КБ)	**10–24**	**до 100**

		3–5 (объем данных до 256 КБ)		
AMD Ryzen 7 7700	**1**	**1–2**	**2–3** (объем данных до 16 МБ) **3–10** (объем данных до 32 МБ)	**до 16**

Задание

Разработать программу для измерения латентности обращения к кэш-памяти различного уровня и оперативной памяти для выбранной вычислительной системы в задаче нахождения суммы элементов вектора длины *N*. Реализовать следующие порядки обхода элементов массива.

1. Последовательный.
2. Случайный.
3. Случайный с использованием дополнительного массива индексов.

Реализовать обработку массивов различного размера, измерить время выполнения одной итерации цикла вычисления суммы с накоплением. Рекомендации:

1. Объем обрабатываемых данных – до 2 МБ, шаг – 1 КБ (измерение латентности доступа в кэши L1d и L2).
2. Объем обрабатываемых данных – до 32 МБ, шаг – 512 КБ (измерение латентности доступа в кэши L2 и L3 (при наличии последнего)).
3. Объем обрабатываемых данных – до 150 МБ, шаг – 5 МБ (измерение латентности доступа в кэши L2/L3/L4 (HBM, при наличии) и оперативную память).

Построить графики зависимости времени выполнения одной итерации цикла вычисления суммы с накоплением от объема обрабатываемых данных. На их основе сделать выводы о латентности доступа к кэш-памяти различного уровня и к оперативной памяти для выбранной вычислительной системы, полученные значения латентности представить в табличном виде. Сравнить полученные цифры с приведенными выше.